

Label-efficient learning for High-Resolution Land Cover Classification using Large-Scale Self-Supervised Pre-Training and Transfer Learning

Dakota Hester, et al.

February 13, 2025

Abstract

Land cover data is a key component of environmental analysis and informs decision making in agriculture, urban planning, hydrology, and other applied domains. Most publicly available land cover products are produced at a coarse resolution due to the inherent difficulty in classifying high resolution data. Deep learning semantic segmentation models have been shown to be performant for such tasks but require large amounts of fully annotated tiles for training. This is problematic for land cover classification due to the expense and difficulty of annotating such data at scale. In our approach, we initially pre-train a ResNet-152 model on 377,913 unlabeled images using a denoising autoencoder to enable spatial feature extraction without the need for labeled data. This model is used as an encoder in a U-Net architecture where fully supervised pre-training is performed using 10,220 labeled images from another region. Finally, we fine-tune the U-Net with only 119 labeled images over the state of Mississippi. When applied to 1-meter imagery in Mississippi, our model achieves over 80

1 Introduction

2 Related Works

3 Data

3.1 National Agriculture Imagery Program (NAIP)

3.2 Chesapeake Bay Land Use and Land Cover

4 Methodology

4.1 Sampling

Ideally, land cover classification models should be trained and validated against samples that are representative of the study area. However, under the constraints of limited labeled data, it may be difficult to capture the full range of variation in the study area. Stratified sampling is a common approach to address this issue, where the study area is divided into strata based on some characteristic (such as land cover class or sub-region) and samples are drawn from each stratum. Unlike traditional land cover classification models, our semantic segmentation model necessitates the use of densely annotated tiles that exhibit a high degree of diversity with regards to spatial characteristics. Ideally, a single training tile should consist of a mix of land cover classes, and the arrangement of land cover classes should similarly be representative of the study area as a whole. To achieve this, we employ a stratified sampling strategy using the National Land Cover Database (NLCD) to guide the selection of training, validation, and testing tiles. This ensures that both the distribution of classes in the tiles is representative of the study area and the content of the tiles is heterogeneous and diverse in terms of spatial characteristics.

The NLCD provides land cover data at a 30m resolution for the entire United States, and the most recent edition (Annual NLCD) provides yearly land cover data between 1985 and 2023. We use 2023 NLCD data to ensure the land cover data is consistent with the NAIP imagery acquired during the same year. For our land cover product, we reclassify the level 1 land cover product to correspond with our choice of legend according to the mapping schema in Table ??.

A grid of 256 m x 256 m square tiles is overlaid over the study area, and the proportion of each land cover class is calculated for each cell. Applying principal component analysis (PCA) to the land cover relative frequency data, we identify that the first two principal components capture 94.24% of the data variance. Figure ?? visualizes the first two principal components of the land cover data at the tile level. Higher values of the first principal component correspond to a higher proportion of impervious land cover types in each tile, while higher values of the second principal component correspond to a higher proportion of forested and herbaceous land cover, with lower values correlating with regions that consist of tiles primarily or wholly composed of cultivated crops. These tiles are subsequently grouped into 250 clusters using K-means clustering operating on these two principal components. From each cluster, we sample 2 candidate tiles for training, 1 for validation, and 1 for testing. This results in a total of 500 training tiles, 250 validation tiles, and 250 testing tiles.

Ensuring this strategy yields high-quality training data that is representative of the study area is crucial to the success of the model. However, we recognize that a rigid evaluation of such an approach to stratification is not possible due to the lack of ground truth data for the study area. One measure we take to ensure the stratification is effective is to inspect the distribution of land cover classes across the splits. Figure ?? shows that the distribution of land cover classes is consistent across the training, validation, and testing splits, which is ideal for ensuring that the model is exposed to a representative sample of the study area’s land cover diversity. One important note is that the distribution of land cover classes in the study area is highly imbalanced, with the majority of Mississippi’s land cover being classified as forest, and a low density of urban and developed areas. To tackle this issue, other strategies often employ synthetic oversampling of minority classes or strategic undersampling of majority classes [CITE](#). Although our approach to stratifying the data does not explicitly enforce such policies, comparing the land cover class distribution in the sampled tiles to the distribution across the entire state (as shown in Figure ??) demonstrates that our sampling strategy achieves a similar outcome to these oversampling and undersampling strategies. Furthermore, our approach tends to avoid sampling tiles that are composed of a single land cover class. In the original NLCD data, 29.96% of the tiles are composed of a single land cover class, while only 1.90% of the sampled tiles are composed of a single land cover class. This is promising, as these homogeneous tiles are likely to be less informative for training the model [CITE](#).

4.2 Model configuration

Our model is based on the U-Net architecture, with three important modifications. First, in lieu of the original convolutional encoder network in the original implementation, we employ a much deeper ResNet-152 network as the encoder. Our reasoning for this choice is that the increased number of layers in the ResNet-152 network will facilitate more complex feature extraction, particularly during the self-supervised pre-training phase, where the large amount of unlabeled data will allow the encoder to learn to capture rich representations of spatial features in the input imagery. Second, we replace the transposed convolutional layers in the original U-Net decoder with bilinear upsampling to reduce the presence of checkerboard artifacts in the upsampled feature maps, which can inhibit the quality of the output segmentation maps [?, ?]. Finally, in each decoder block, we introduce a residual connection between the output of the first convolutional layer and the output of the second convolutional layer to facilitate the flow of information through the network and improve training stability [?].

4.3 Pre-training

Our pre-training approach consists of two phases: a self-supervised pre-training phase and a fully supervised pre-training phase.

4.3.1 Self-supervised pre-training

In the first stage, we construct a simple denoising convolutional autoencoder by resampling feature maps from each stage of the ResNet-152 encoder to the original input resolution, concatenating them, and passing them through a single 1×1 convolutional layer to produce an output with the same dimensions as the input. As the decoder network only consists of a single layer, the encoder network is primarily responsible for robust feature extraction in this phase.

Two types of noise are added to the image: Gaussian noise and Poisson noise. The intensity of the Gaussian noise is controlled by the standard deviation parameter σ , which is randomly sampled from a uniform distribution $U(0, 2)$. Similarly, the intensity of the Poisson noise is controlled by the rate parameter λ , which is also randomly sampled from a uniform distribution $U(0, 2)$.

Our implementation is distinct from other denoising autoencoders in that the amount and type of noises added to an image is random, and not a fixed amount **CITE!!!**. Ideally, while this may inhibit the model’s ability to effectively denoise an image, it will force the model to learn robust features that are invariant to the type and amount of noise present in the input image **CITE!!!!???**.

During this phase

4.3.2 Fully supervised pre-training

Once the

4.4 Fine-tuning

4.5 Inference

4.6 Evaluation

5 Preliminary Results

6 Conclusion